



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Processamento de Linguagens

Ano Letivo de 2025/2026

Projeto Processamento de Linguagens

Filipe Viana	Pedro Argainha	João Nuno
a104361	a104351	a104084

PL

Data da Receção	
Responsável	
Avaliação	
Observações	

Projeto Processamento de Linguagens

maio, 2026

Índice

1. Introdução	4
2. Análise Léxica	5
2.1. Palavras Reservadas	5
2.2. Tokens e Operadores	5
2.3. Tratamento de Comentários	5
3. Análise Sintática	7
3.1. Gramática	7
3.2. Precedência de Operadores	7
3.3. Tabela de Símbolos	8
4. Análise Semântica	9
5. Geração de Código	10
5.1. Variáveis e Atribuição	10
5.2. Arrays	10
5.3. Controlo de Fluxo	10
5.4. Input/Output	11
5.5. Subprogramas (FUNCTION) – Valorização	11
5.6. SUBROUTINE e CALL	12
5.7. Funções Intrínsecas	12
6. Testes e Resultados	13
6.1. Exemplo 1: Olá, Mundo!	13
6.2. Exemplo 2: Fatorial	13
6.3. Exemplo 3: É Primo?	13
6.4. Exemplo 4: Soma de Array	13
6.5. Exemplo 5: Conversor de Bases (FUNCTION)	13
6.6. Exemplo 6: SUBROUTINE (teste adicional)	14
6.7. Exemplo 7: Fibonacci (teste adicional)	14
6.8. Exemplo 8: Tabuada (teste adicional)	15
6.9. Exemplo 9: Operações Lógicas (teste adicional)	15
6.10. Exemplo 10: MAX/MIN com Arrays (teste adicional)	15
6.11. Resumo dos Testes	16
7. Tratamento de Erros	17
8. Dificuldades Encontradas	18
9. Instruções de Execução	19
9.1. Pré-requisitos	19
9.2. Compilação	19
9.3. Execução na EWVM	19
10. Conclusão	20

1. Introdução

O presente relatório descreve o desenvolvimento de um compilador para um subconjunto da linguagem **Fortran 77** (standard ANSI X3.9-1978), realizado no âmbito da Unidade Curricular de Processamento de Linguagens (PL), ano letivo 2025/2026.

O compilador foi implementado em **Python 3**, utilizando a biblioteca **PLY** (*Python Lex-Yacc*) para a construção do analisador léxico e do analisador sintático. A máquina-alvo é a **EWVM** (*Educational Web Virtual Machine*), disponibilizada em <https://ewvm.epl.di.uminho.pt/>.

O compilador é capaz de processar e traduzir programas Fortran 77 que incluam:

- Declaração de variáveis (tipos `INTEGER`, `REAL`, `LOGICAL`) e arrays unidimensionais;
- Expressões aritméticas, lógicas e relacionais;
- Comandos de controlo de fluxo (`IF-THEN-ELSE`, ciclos `DO` com labels, `GOTO`);
- Operações de Input/Output (`READ`, `PRINT`);
- Funções intrínsecas (`MOD`, `ABS`, `MAX`, `MIN`);
- Definição e chamada de subprogramas (`FUNCTION` e `SUBROUTINE`) como valorização.

O formato adotado para o código-fonte Fortran é o **free-form** (formato livre), em contraste com o formato de colunas fixas do Fortran 77 clássico.

2. Análise Léxica

A análise léxica converte o código-fonte Fortran numa sequência de *tokens* reconhecidos pelo parser. Foi implementada no ficheiro `fortran_lexer.py` usando o módulo `ply.lex`.

2.1. Palavras Reservadas

As seguintes palavras-chave são reconhecidas, de forma *case-insensitive*:

PROGRAM	END	INTEGER	REAL	LOGICAL
IF	THEN	ELSE	ENDIF	DO
CONTINUE	GOTO	READ	PRINT	STOP
FUNCTION	RETURN	CALL	SUBROUTINE	

Tabela 1: Palavras reservadas reconhecidas pelo lexer.

A regra de identificação usa `t.value.upper()` para normalizar os identificadores e, caso o resultado esteja no dicionário `reserved`, classifica-o como keyword; caso contrário, como `ID`.

2.2. Tokens e Operadores

Categoria	Tokens	Exemplo
Literais	INTLIT, REALLIT, STRLIT	42, 3.14, 'hello'
Lógicos	.TRUE., .FALSE.	.TRUE.
Relacionais	.EQ., .NE., .LT., .GT., .LE., .GE.	.EQ.
Lógicos	.AND., .OR., .NOT.	.AND.
Aritméticos	+, -, *, /	+
Delimitadores	(,), ,, =	(
Controlo	NEWLINE	separador de <i>statements</i>

Tabela 2: Categorias de tokens reconhecidos.

2.3. Tratamento de Comentários

No formato *free-form*, os comentários são marcados pelo carácter `!` e estendem-se até ao final da linha. O lexer trata-os com a regra:

```
def t_COMMENT(t):  
    r'![^\n]*'  
    pass # Ignorar comentários inline
```

O Fortran 77 clássico trata linhas começadas por `C`, `c` ou `*` como comentários. No nosso compilador, esta funcionalidade foi deliberadamente desativada para evitar conflitos com variáveis cujos nomes começam por `C` (como `CONVRT`, `CONTINUE`), uma vez que utilizamos o formato *free-form*.

3. Análise Sintática

O analisador sintático foi implementado no ficheiro `fortran_parser.py` usando `ply.yacc`, seguindo uma gramática LALR(1). A abordagem adotada foi a **tradução dirigida pela sintaxe**: o código EWVM é gerado diretamente nas ações semânticas das regras gramaticais, sem construção de uma AST intermédia.

3.1. Gramática

A gramática principal do compilador pode ser resumida nas seguintes produções:

```
program      → PROGRAM ID NL declarations statements END NL subprograms
declarations → declarations declaration NL | ε
declaration  → tipo var_list
var_list     → var_decl | var_list COMMA var_decl
var_decl     → ID | ID(INTLIT)
statements   → statements statement NL
              | statements labeled_statement NL
              | statements NL | ε
statement    → assignment | if_stmt | do_stmt | goto
              | print_stmt | read_stmt | stop
labeled_stmt → INTLIT statement
if_stmt      → IF (expr) THEN NL stmts ENDIF
              | IF (expr) THEN NL stmts ELSE NL stmts ENDIF
do_stmt      → DO INTLIT ID = expr, expr
goto         → GOTO INTLIT
subprogram   → func_header NL decls stmts RETURN NL END
func_header  → INTEGER FUNCTION ID (param_list)
expression   → term ((+|-) term)*
term         → factor ((*|/) factor)*
factor       → INTLIT | STRLIT | ID | ID(expr) | (expr)
              | .TRUE. | .FALSE. | .NOT. factor | -factor
              | ID(expr, expr)
```

3.2. Precedência de Operadores

A precedência dos operadores foi definida explicitamente para resolver ambiguidades na gramática:

```
precedence = (
    ('left', 'OR'),
    ('left', 'AND'),
    ('right', 'NOT'),
    ('nonassoc', 'EQ', 'NE', 'LT', 'GT', 'LE', 'GE'),
    ('left', 'PLUS', 'MINUS'),
    ('left', 'TIMES', 'DIVIDE'),
    ('right', 'UMINUS'),
)
```

3.3. Tabela de Símbolos

A classe `SymbolTable` gere os identificadores do programa. Suporta:

- **Variáveis globais:** indexadas sequencialmente a partir de 0, acessíveis via `PUSHG` / `STOREG` ;
- **Arrays:** alocação contígua no espaço global, acesso via `PUSHGP` / `PADD` / `LOADN` ;
- **Variáveis locais** (em funções): índices positivos relativos ao *frame pointer* (`PUSHL` / `STOREL`);
- **Parâmetros de função:** mapeados para índices negativos relativos ao FP.

```
class SymbolTable:
    def __init__(self):
        self.globals = {}      # nome → {type, index, is_array}
        self.locals = {}      # nome → {type, index}
        self.functions = {}    # nome → {type, params}
        self.in_subprogram = False
```


4. Análise Semântica

A análise semântica encontra-se integrada diretamente nas regras do parser (tradução dirigida pela sintaxe). As verificações realizadas incluem:

- **Declaração de variáveis:** Todas as variáveis devem ser declaradas antes de serem utilizadas. A tabela de símbolos regista o tipo, índice e, no caso de arrays, o tamanho;
- **Escopo:** Dentro de uma `FUNCTION`, o compilador distingue entre variáveis locais e globais, consultando primeiro a tabela local;
- **Tipos:** O sistema reconhece os tipos `INTEGER`, `REAL` e `LOGICAL`, gerando instruções diferentes conforme o tipo (ex: `WRITEI` vs `WRITES`);
- **Labels:** Os labels numéricos usados em `DO / CONTINUE` e `GOTO` são mapeados para labels internos (`L1`, `L2`, ...) e verificados quanto à correspondência;
- **Função intrínseca:** A função `MOD(a, b)` é reconhecida e traduzida diretamente para a instrução `MOD` da EWVM.

5. Geração de Código

A geração de código traduz as construções Fortran para instruções da máquina virtual EWVM. A tradução é feita diretamente nas regras `yacc`.

5.1. Variáveis e Atribuição

A atribuição `X = expr` gera código para avaliar a expressão (empilhando o resultado) seguido de `STOREG n` (global) ou `STOREL n` (local):

Fortran	EWVM
<code>FAT = 1</code>	<code>PUSHI 1</code> <code>STOREG 2</code>
<code>FAT = FAT * I</code>	<code>PUSHG 2</code> <code>PUSHG 1</code> <code>MUL</code> <code>STOREG 2</code>

Tabela 3: Tradução de atribuições simples.

5.2. Arrays

Os arrays Fortran são *1-based*. O acesso `NUMS(I)` é traduzido para:

```
PUSHGP      // endereço base global
PUSHI offset // offset do array
PADD        // endereço base do array
PUSHG i      // valor de I
PUSHI 1
SUB          // I - 1 (conversão para 0-based)
LOADN       // carregar valor
```

5.3. Controlo de Fluxo

5.3.1. IF-THEN-ELSE

A construção `IF (cond) THEN ... ELSE ... ENDIF` traduz-se em:

```

<código da condição>
JZ else_label
<código do bloco THEN>
JUMP end_label
else_label:
<código do bloco ELSE>
end_label:

```

5.3.2. DO Loop

O ciclo `DO n I = start, end ... n CONTINUE` gera:

```

<push start>           // valor inicial
STOREG i               // I = start
loop_label:
PUSHG i                // push I
<push end>             // push end
SUP                    // I > end?
NOT                    // !(I > end) = I <= end
JZ end_label           // se falso, sai
<corpo do ciclo>
NOP                    // CONTINUE
PUSHG i                // incremento: I = I + 1
PUSHI 1
ADD
STOREG i
JUMP loop_label
end_label:

```

5.3.3. GOTO

O comando `GOTO n` traduz-se simplesmente em `JUMP Ln`, onde `Ln` é o label EWVM correspondente ao label Fortran `n`.

5.4. Input/Output

- `READ *, X` → `READ / ATOI / STOREG n`
- `PRINT *, items` → Para cada item: strings usam `PUSHS / WRITES`, inteiros usam `PUSHG / WRITEI`, e no final é emitido `WRITELN`.

5.5. Subprogramas (FUNCTION) — Valorização

A implementação de `FUNCTION` segue uma convenção de chamada adaptada à EWVM:

5.5.1. Convenção de Chamada

Chamador (caller):

1. Empilha os argumentos (`PUSHG`)
2. `PUSHA fname / CALL`
3. Após `RETURN`, faz `POP n_args` para limpar argumentos da stack
4. Lê o resultado de uma variável global dedicada (`PUSHG __ret_fname`)

Função (callee):

1. `PUSHN n_locals` — aloca variáveis locais
2. Acede a parâmetros via `PUSHL -n` (offsets negativos ao FP)
3. Acede a locais via `PUSHL n` (offsets positivos ao FP)
4. No final, copia o valor de retorno para um global dedicado (`STOREG`)
5. `POP n_locals` — limpa variáveis locais (SP volta a FP)
6. `RETURN` — restaura FP e PC

Fortran	EWVM (caller)
<code>RESULT = CONVRT(NUM, BASE)</code>	<code>PUSHG 0 (NUM)</code> <code>PUSHG 1 (BASE)</code> <code>PUSHA CONVRT</code> <code>CALL</code> <code>POP 2</code> <code>PUSHG 4 (__ret_CONVRT)</code> <code>STOREG 2 (RESULT)</code>

Tabela 4: Tradução de uma chamada de função.

Esta abordagem utiliza uma variável global como canal de retorno em vez de manipulação complexa da stack, evitando limitações da EWVM (que não permite `POP` abaixo do *frame pointer*).

5.6. SUBROUTINE e CALL

O compilador suporta também `SUBROUTINE`, subprogramas sem valor de retorno. A instrução `CALL nome(args)` empilha os argumentos, executa `PUSHA / CALL`, e após o `RETURN` da subroutine faz `POP n_args` para limpar a stack.

Fortran	EWVM
<code>CALL IMPRIME(A, B)</code>	<code>PUSHG 0 (A) \ PUSHG 1 (B) \ PUSHA IMPRIME \ CALL \ POP 2</code>

Tabela 5: Tradução de uma chamada CALL.

Dentro da subroutine, os parâmetros são acedidos via `PUSHL -n` (offsets negativos). No epílogo, faz-se `POP n_locals` seguido de `RETURN`.

5.7. Funções Intrínsecas

Além de `MOD(a, b)` (traduzido para a instrução `MOD` da EWVM), o compilador suporta:

- **ABS(x)** : valor absoluto, implementado com `DUP 1`, teste de sinal, e multiplicação condicional por `-1`;
- **MAX(a, b)** e **MIN(a, b)** : avalia ambos os argumentos, compara com `SUP / INF` (consumindo os valores), e depois re-avalia o argumento vencedor.

6. Testes e Resultados

O compilador foi testado com os 5 exemplos fornecidos no enunciado do projeto, mais 5 testes adicionais. Todos compilam corretamente e produzem os resultados esperados na EWVM.

6.1. Exemplo 1: Olá, Mundo!

```
PROGRAM HELLO
PRINT *, 'Ola, Mundo!'
END
```

Listagem 1: Código-fonte `hello.f`.

Saída na EWVM: `Ola, Mundo!`

6.2. Exemplo 2: Fatorial

Resultado para input `5`: `Fatorial de 5: 120` ✓

6.3. Exemplo 3: É Primo?

Usa a construção `20 IF (cond .AND. ISPRIM) THEN ... GOTO 20 ENDIF` para iterar. Resultado para input `7`: `7 e um numero primo` ✓. Para input `4`: `4 nao e um numero primo` ✓.

6.4. Exemplo 4: Soma de Array

Lê 5 números inteiros e calcula a soma. Para inputs `1, 2, 3, 4, 5`: `A soma dos numeros e: 15` ✓.

6.5. Exemplo 5: Conversor de Bases (FUNCTION)

Utiliza a função `CONVRT(N, B)` para converter um número decimal nas bases 2 a 9.

Resultado para input `42`:

```

INTRODUZA UM NUMERO DECIMAL INTEIRO:
Base 2: 101010
Base 3: 1120
Base 4: 222
Base 5: 132
Base 6: 110
Base 7: 60
Base 8: 52
Base 9: 46

```

Listagem 2: Saída do programa Conversor na EWVM.

6.6. Exemplo 6: SUBROUTINE (teste adicional)

```

PROGRAM TESTSUB
INTEGER A, B, C
PRINT *, 'Introduza dois numeros:'
READ *, A
READ *, B
CALL IMPRIME(A, B)
C = A + B
PRINT *, 'Soma: ', C
END

SUBROUTINE IMPRIME(X, Y)
INTEGER X, Y
PRINT *, 'Valor 1: ', X
PRINT *, 'Valor 2: ', Y
RETURN
END

```

Listagem 3: Código-fonte `subroutine.f`.

Saída para inputs 7 e 3: Valor 1: 7, Valor 2: 3, Soma: 10 ✓

6.7. Exemplo 7: Fibonacci (teste adicional)

```

PROGRAM FIBONACCI
INTEGER N, I, A, B, TEMP
PRINT *, 'Quantos termos de Fibonacci?'
READ *, N
A = 0
B = 1
DO 10 I = 1, N
    PRINT *, A
    TEMP = A + B
    A = B
    B = TEMP
10 CONTINUE
END

```

Listagem 4: Código-fonte `fibonacci.f`.

Saída esperada para input 8: 0, 1, 1, 2, 3, 5, 8, 13 ✓

6.8. Exemplo 8: Tabuada (teste adicional)

O utilizador introduz um número e o programa imprime a tabuada desse número até $\times 12$.

```
PROGRAM TABUADA
INTEGER N, I
PRINT *, 'Introduza um numero (1-12):'
READ *, N
DO 10 I = 1, 12
    PRINT *, N, ' x ', I, ' = ', N * I
10 CONTINUE
END
```

Listagem 5: Código-fonte `tabuada.f`.

Saída para input 5: 5 x 1 = 5, 5 x 2 = 10, ..., 5 x 12 = 60 ✓

6.9. Exemplo 9: Operações Lógicas (teste adicional)

Testa variáveis `LOGICAL`, operadores `.GT.`, `.EQ.`, `.AND.` e `IF-THEN-ELSE` aninhados.

```
PROGRAM LOGICA
INTEGER X
LOGICAL POSITIVO, PAR, AMBOS
READ *, X
POSITIVO = X .GT. 0
PAR = MOD(X, 2) .EQ. 0
AMBOS = POSITIVO .AND. PAR
IF (AMBOS) THEN
    PRINT *, X, ' e positivo e par'
ELSE
    IF (POSITIVO) THEN
        PRINT *, X, ' e positivo mas impar'
    ENDIF
ENDIF
END
```

Listagem 6: Código-fonte `logica.f` (simplificado).

6.10. Exemplo 10: MAX/MIN com Arrays (teste adicional)

Utiliza as funções intrínsecas `MAX` e `MIN` para encontrar o maior e menor valor num array.

```

PROGRAM MAXMIN
INTEGER NUMS(5), I, VMAX, VMIN
DO 10 I = 1, 5
    READ *, NUMS(I)
10 CONTINUE
VMAX = NUMS(1)
VMIN = NUMS(1)
DO 20 I = 2, 5
    VMAX = MAX(VMAX, NUMS(I))
    VMIN = MIN(VMIN, NUMS(I))
20 CONTINUE
PRINT *, 'Maximo: ', VMAX
PRINT *, 'Minimo: ', VMIN
END

```

Listagem 7: Código-fonte `maxmin.f` (simplificado).

6.11. Resumo dos Testes

#	Programa	Funcionalidades testadas	Resultado
1	<code>hello.f</code>	PRINT, strings	✓
2	<code>fatorial.f</code>	READ, DO loop, aritmética	✓
3	<code>primo.f</code>	LOGICAL, .AND., GOTO, IF aninhado	✓
4	<code>somaarr.f</code>	Arrays, DO, READ em array	✓
5	<code>conversor.f</code>	FUNCTION, CALL, retorno, MOD	✓
6	<code>subroutine.f</code>	SUBROUTINE, CALL com args	✓
7	<code>fibonacci.f</code>	DO loop, múltiplas atribuições	✓
8	<code>tabuada.f</code>	DO loop, READ, aritmética	✓
9	<code>logica.f</code>	LOGICAL vars, IF-ELSE aninhado	✓
10	<code>maxmin.f</code>	MAX, MIN, arrays	✓

Tabela 6: Resumo dos 10 programas de teste.

7. Tratamento de Erros

O compilador implementa detecção e relatório de erros a três níveis, acumulando-os numa lista e apresentando um relatório consolidado no final da compilação.

Os tipos de erros detetados são:

- **Erros léxicos:** caracteres não reconhecidos pela gramática (ex: @, #, \$);
- **Erros sintáticos:** tokens inesperados que violam as regras gramaticais (ex: expressão incompleta, parêntesis em falta);
- **Erros semânticos:** utilização inválida de identificadores (ex: acesso a um array não declarado).

O compilador não termina ao encontrar o primeiro erro — continua a análise para reportar o máximo de problemas possível numa única execução.

Exemplo de saída para um programa com múltiplos erros:

```
=== RELATÓRIO DE ERROS ===
  1. Erro léxico na linha 4: carácter inesperado '@'
  2. Erro léxico na linha 4: carácter inesperado '#'
  3. Erro sintático na linha 4: token inesperado (tipo NEWLINE)

Total: 3 erro(s) encontrado(s)
```

8. Dificuldades Encontradas

Durante o desenvolvimento do compilador, várias dificuldades foram enfrentadas:

- **Preprocessamento vs. Lexing:** A remoção de comentários no estilo Fortran 77 (linhas começadas por `C`) conflitava com identificadores como `CONVRT`. A solução foi adotar o formato *free-form* e usar apenas `!` para comentários;
- **CALL/RETURN na EWVM:** A EWVM não limpa a stack automaticamente no `RETURN` e proíbe `POP` abaixo do *frame pointer*. Foi necessário desenvolver uma convenção de chamada personalizada que usa uma variável global dedicada para o valor de retorno;
- **DO loops com CONTINUE:** A correspondência entre labels numéricas Fortran (`DO 10 ... 10 CONTINUE`) e labels EWVM exigiu um mapeamento cuidadoso e a geração do código de incremento na regra do `labeled_statement`;
- **Conflitos LALR:** A regra `statement : RETURN` consumia o token `RETURN` antes da regra de `FUNCTION` o poder ver. A solução foi remover essa regra redundante, permitindo que apenas a regra do subprograma consuma o `RETURN`.

9. Instruções de Execução

9.1. Pré-requisitos

- Python 3.x
- Biblioteca PLY: `pip install ply`

9.2. Compilação

```
python3 compiler.py <ficheiro.f>
```

O código EWVM é impresso no *standard output*. Para guardar num ficheiro:

```
python3 compiler.py tests/hello.f > tests/hello.vm
```

9.3. Execução na EWVM

Copiar o conteúdo do ficheiro `.vm` e colar na interface web da EWVM em <https://ewvm.epl.di.uminho.pt/>, clicando em *Run*.

10. Conclusão

O compilador desenvolvido cumpre todos os requisitos obrigatórios definidos no enunciado do projeto: análise léxica, análise sintática, análise semântica e geração de código para a EWVM. Adicionalmente, foram implementadas as funcionalidades de valorização com o suporte a subprogramas do tipo `FUNCTION` e `SUBROUTINE`, bem como funções intrínsecas adicionais (`ABS`, `MAX`, `MIN`).

Os 5 programas de exemplo do enunciado — incluindo o conversor de bases com chamada de função — compilam e executam corretamente na máquina virtual EWVM. Foram também criados 5 testes adicionais para validar funcionalidades como `SUBROUTINE`, `DO` loops aninhados, operações lógicas, sequência de Fibonacci, e funções `MAX` / `MIN` com arrays.

O compilador utiliza uma abordagem de tradução dirigida pela sintaxe, gerando código diretamente nas ações semânticas das regras `yacc`. Esta abordagem, embora menos modular do que a construção de uma AST intermédia, revelou-se eficaz e pragmática para o subconjunto de Fortran 77 considerado, resultando num projeto compacto de apenas 3 ficheiros-fonte (lexer, parser e compilador).